Mindscape



Charbel Ghanem, Charbel Khalifeh and Joe Obeid

Present

Lightning McQueen's Engineering Journal

A WRO Future Engineers Project

July 2026

Table of Contents

Introduction 5
Mobility & Mechanical Design 5
Version 1 5
Problems Identified 6
Final Design 6
1.1 Components 7
Power & Sensor Architecture 7
Compute 7
Camera 8
Distance Sensing 8
Power Supply 8
Thermal Management 9
Supporting Circuit 9
Wirings 9
Software Architecture & Obstacle Strategy 10
Open Challenge - State Machine 10
Wall-Centering (Primary Navigation) 10
Corner Detection & Direction Logic 10
Turn Execution 10
Front-Wall Safety Interrupt 11
Stuck Detection 11
Start/Stop & Tuning 11
Obstacle Challenge — Overall Strategy 12
Region-of-Interest Based Detection 12
Traffic Sign Recognition and Avoidance 13
Dynamic State Transitions 13
Parking Procedure 13
Systems Thinking & Engineering Decisions 15
Diagnosing Hardware Problems Instead of Patching in Software 15
Isolating Power Rails by Design 15
Camera as Primary Sensor 15
Resuming Interrupted Turns Instead of Restarting Them 15
Constraints 17
The Wheel Material Decision 17
Failure Modes Encountered 17
Reproducibility & GitHub Repository 18

Testing Log 19
Camera Color Calibration 19
Attempt 1 — Dynamic Line-Fitting 19
Attempt 2 — Early ROI Strategy + Camera FOV Investigation 19
Attempt 3 — Dual-Ultrasonic PID Wall-Following 20
After the Redesign — Returning to the ROI Strategy 20
Obstacle Challenge Development 20
Transition to the ROI-Based Strategy 21
Incremental Development 21
Steering Mechanism Failure 21
Final Validation 21
Appendix A 22
Robot Photos 22
Appendix B: Wiring Diagram 22
Appendix C: Video Links 23

Introduction

This journal documents the engineering process behind Lightning McQueen, the autonomous vehicle we built for the WRO 2026 Future Engineers Self-Driving Cars challenge. Rather than only presenting our final robot, this document tells the story of how it evolved: the problems we ran into with our first build, the decisions we made to solve them, and the reasoning behind our final design and code architecture.

Charbel Ghanem (19) – A second-year Mechatronics Engineering student at the Lebanese American University (LAU). This is his second WRO competition. In WRO 2025, his team placed 3rd in the Future Engineers category. This year, he contributed to the CAD design and served as the team's main programmer.

Charbel Khalife (16) — A secondary school student with a strong interest in robotics and programming. This is his third WRO competition. He previously competed in the Junior RoboMission category and earned 2nd place in the WRO 2025 Future Engineers category. He was responsible for the team's documentation and also contributed to the robot's software development.

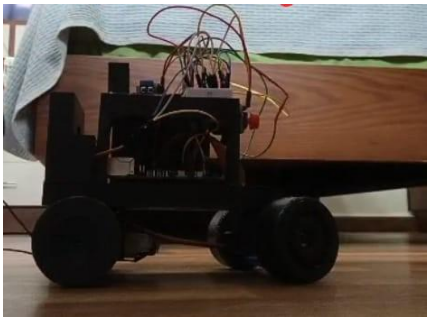
Joe Obeid (22) – A fifth-year Mechanical Engineering student at CNAM University. This is his first WRO competition. Joe was primarily responsible for designing the robot's CAD models and manufacturing the parts using 3D printing.

Mobility & Mechanical Design

Version 1

Our first build used a compact chassis with the camera mounted on a front tower, a chamber on the main deck holding the breadboard and motor driver, and the power bank placed in a slot at the back of the robot, on top of that chamber. The Raspberry Pi 5 was housed alone in its own chamber. Steering was handled by a micro servo through a plastic linkage, with a single DC motor driving a solid rear axle.







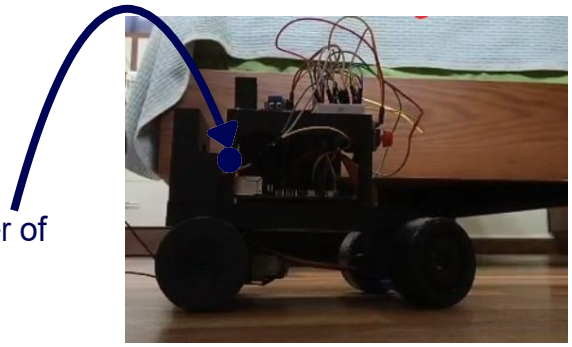
Problems Identified

Weight distribution. The powerbank sat high and at the very back of the chassis. This pushed the center of gravity up and back, and the robot leaned noticeably in turns, occasionally lifting a wheel off the ground and required a large power to accelerate, costing us both speed and reliability. This was the main issue driving our redesign.

Camera field of view. The camera itself has a limited field of view, and mounting it at the front of the robot compounded the issue, leaving blind spots that caused obstacles to be missed. We attempted
to fix this by repositioning and adjusting the camera without changing the rest of the design, but could not reach a clean fix this way.

Wheel grip. Our first wheels were standard, rigid PLA prints with poor grip that wore down quickly under turning loads.

The robot weighed aprx. 1.2 kg and with an elevated center of mass at the back



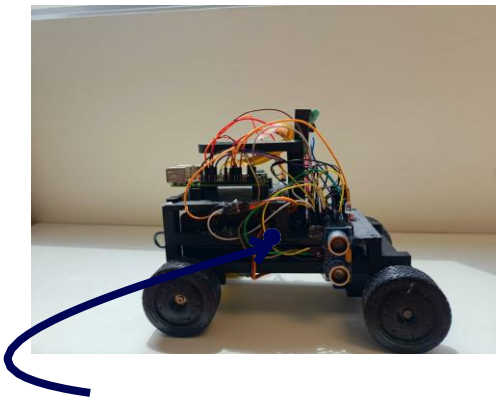
Final Design

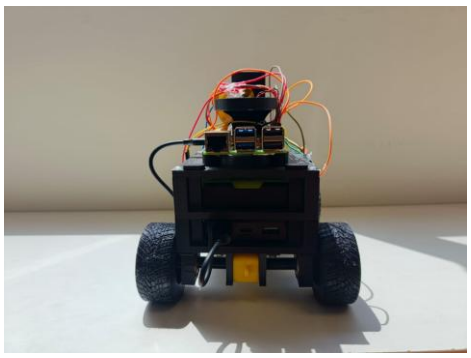
Sliding battery/powerbank plate. The batteries and powerbank now sit stacked on top of each other on a sliding plate, positioned lower and more centrally in the chassis. This makes them easy to pull out and swap, and solves the weight distribution problem identified in Version 1.

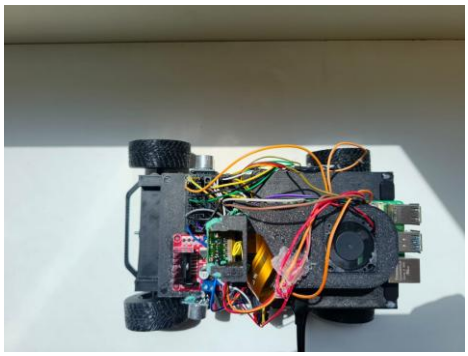
Improved cooling. The fan now sits on top of the Raspberry Pi 5 instead of underneath it, improving
airflow and cooling.

Repositioned camera. The camera was moved back slightly and mounted higher on the tower.

New wheels. We reprinted the wheels using PLA filament printed at a higher nozzle temperature than standard settings, which produced a softer, more rubber-like surface with noticeably better grip.







1.1 Components

DC Motor (yellow plastic gearbox motor).

- Voltage Range: 3V to 6V DC
- No-Load Current: ≈ 150 mA at 3V, up to 200 mA at 6V
- No-load Speed (at 6V): ≈ 200 RPM (± 10%)
- Stall Torque (at 6V): ≈ 0.8 kg-cm

Servo, SG90.

- Operating Voltage: 4.8V - 6.0V
- Operating Speed: 0.1 s / 60° (at 4.8V, no load)
- Weight: 9 g
- Dimensions: 22.2 × 11.8 × 31 mm
- Gear Type: Nylon/Plastic (though metal-geared variants exist)
- Rotation: 180° standard

Check our Github Repository for the full list:
https://github.com/CharbelKhalife/Mindscape-WRO2026-FUTURE-ENG/tree/master

Power & Sensor Architecture

Compute

The vehicle's controller is a Raspberry Pi 5 (8GB). We started on the 4GB model but switched to the 8GB version after finding the 4GB board too slow, causing noticeable lag during camera processing. It reads the camera and ultrasonic sensors, decides steering/driving commands, and sends them to the actuators described in Section 2. It was chosen for the CPU power needed to run our camera-based wall-percentage detection and traffic-sign recognition in real time, while still fitting within the power budget of the 5V powerbank.

Camera

Navigation relies on a Raspberry Pi Camera v2 (8MP), connected directly to the Pi 5's CSI camera port via ribbon cable. The image feed is processed to estimate what percentage of the frame is taken up by the left wall versus the right wall; the difference between the two is used as the main steering error signal to keep the robot centered between the walls. The same camera is used to detect and classify the red/green traffic-sign pillars in the Obstacle Challenge. 8MP resolution was chosen as enough to reliably estimate wall coverage and pillar color/side at the frame rates needed, without producing more data than the Pi 5 needs to process per frame. In the final design the camera was moved back slightly and mounted higher on the tower to improve this view.

Distance Sensing

Two HY-SRF05 ultrasonic sensors are mounted at different points on the vehicle. These are not used for continuous wall-distance measurement or wall following that's handled by the camera. Instead, they're polled continuously in parallel with the main camera-based control loop as an emergency-braking safety layer: if either sensor reports a distance below a fixed safety threshold, the drive motor is immediately cut/reversed regardless of what the camera-based logic is currently commanding, to prevent a hard collision. They were chosen for being inexpensive, low current draw, and fast enough to act as a reliable last-resort check independent of camera lighting conditions.

Power Supply

Power is split across two isolated rails. A Joyroom 10,000 mAh fast-charging powerbank (rated up to 22.5 W, treated as a hard design constraint) supplies the entire logic side: the Raspberry Pi 5,camera, both ultrasonic sensors, and the supporting LED/button circuit described below. A separate 4x 18650 Li-ion battery pack (4 x 3.7V, ~14.8V nominal, ~16.8V full / ~12V cutoff) supplies the drive side of the vehicle, kept completely isolated from the 5V logic rail to avoid motor current/noise or voltage sag affecting the electronics. The 10,000 mAh power bank was sized to cover a full competition day of practice plus rounds, with a digital display to monitor remaining charge between rounds and fast-charge support to top up quickly during breaks.

Thermal Management

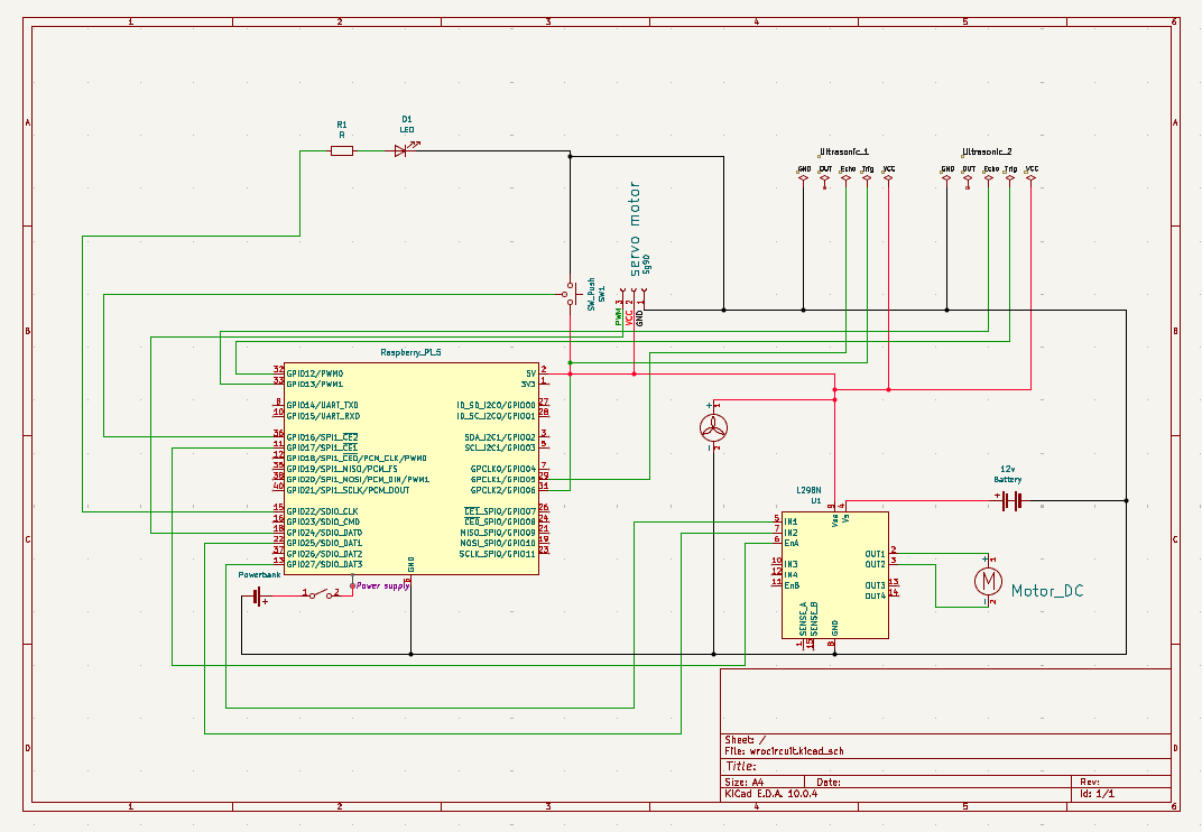
A 5V cooling fan, powered from the logic-side powerbank, provides active cooling for the Raspberry Pi 5. The Pi 5 runs noticeably hotter than previous Pi models under sustained camera-processing load; without active cooling, thermal throttling would slow down frame-processing rate mid-round. In the final design the fan was moved to sit on top of the Pi 5 instead of underneath it, for better airflow.

Supporting Circuit

A status LED (with current-limiting resistor) is driven by a GPIO output and lights up specifically to indicate that the control program is running not just that the vehicle is powered on giving judges and team members an immediate visual check that the program started after the push button is pressed. A push button serves as the mandatory WRO start button. Both are held on a mini breadboard, which let us iterate on this small circuit quickly during development without committing to solder.

Check our Github repository for the full list:
https://github.com/CharbelKhalife/Mindscape-WRO2026-FUTURE-ENG/tree/master

Wirings



Software Architecture & Obstacle Strategy

Open Challenge - State Machine

The Open Challenge run (race.py) is built as a state machine with six states: STRAIGHT (default wall-centering driving), APPROACHING (line seen, steering full-lock toward the turn before committing), TURNING_90 (full-lock turn for a fixed duration), CORNER_BACKUP and REVERSING (wall-avoidance interrupts, described below), and FINISHING (post-lap-12 stop sequence). The robot completes 12 corners (3 laps) before stopping and lighting the finish LED.

Wall-Centering (Primary Navigation)

Rather than following a line, the robot balances itself between the two walls. Two regions of interest are cut from the left and right edges of the camera frame and converted to LAB color space; any pixel darker than a calibrated lightness threshold is counted as “wall.” The difference between the left and right wall-coverage fractions becomes the steering error, which is scaled, clamped to a maximum offset, and smoothed with an exponential filter before being sent to the servo. This keeps the car centered in the lane without needing to see or follow any explicit line.

Corner Detection & Direction Logic

A separate ROI near the bottom of the frame watches for an orange or blue line in HSV color space. The first line color the robot sees for the whole run fixes the turn direction for every remaining corner (orange first → clockwise/right turns, blue first → the opposite) and once that's set, the other color is deliberately ignored for the rest of the run so the robot can't misfire on the “wrong” color later. Seeing a qualifying line moves the state from STRAIGHT to APPROACHING.

Turn Execution

In APPROACHING, the robot steers full-lock toward the fixed turn direction immediately (this delay is tunable but currently set to zero) to help clear the corner. Just before committing to the turn, a small “pre-corner” strip right at the corner is checked; if it's already blocked by wall, the robot reverses first (CORNER_BACKUP) until clear. Otherwise, it enters TURNING_90: full steering lock at a fixed turn speed for a fixed duration this duration is the main tuning parameter for making the robot exit each corner parallel to the next straight.

Front-Wall Safety Interrupt

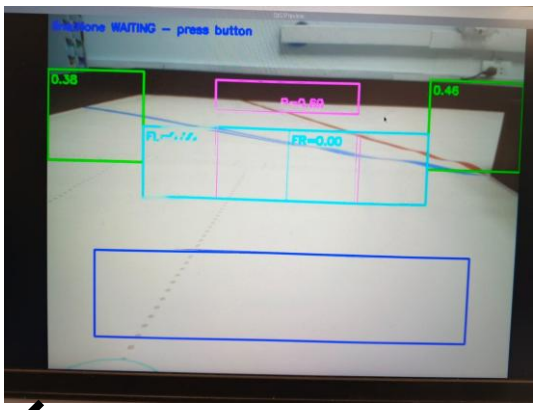
A front-facing ROI, split into left and right halves, is checked continuously regardless of state. If it's triggered while driving straight, the robot reverses straight back if both halves see wall, or reversing while steering away if only one half does. If it's triggered mid-turn, the robot remembers exactly how much turn time was left, backs out of the wall counter-steered, then resumes the turn for the remaining duration only so a wall encountered mid-corner doesn't cost the rest of that turn.

Stuck Detection

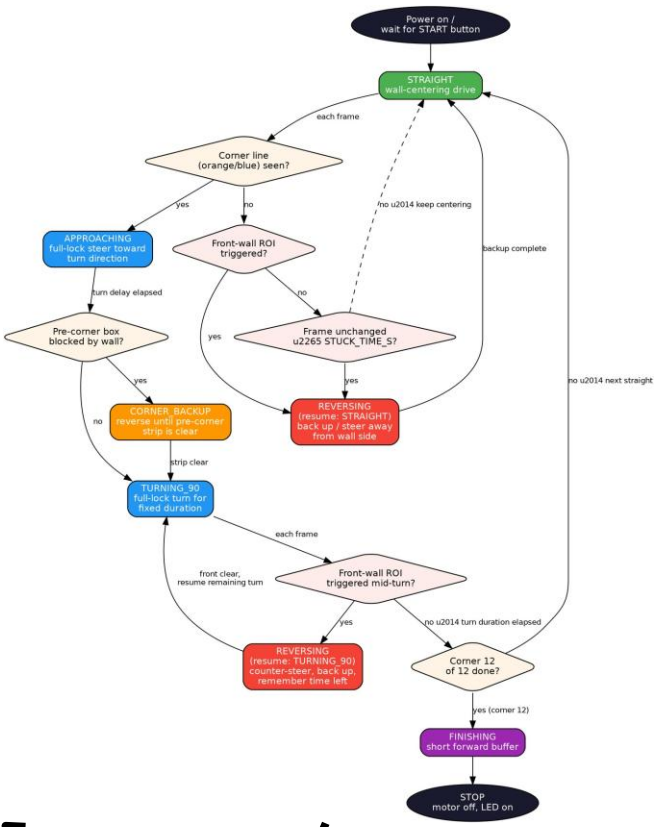
In parallel with the above, consecutive camera frames are compared; if the frame barely changes for a fixed period, the robot assumes it's physically stuck (rather than cleanly blocked by a wall it can react to) and reverses as a catch-all recovery.

Start/Stop & Tuning

The whole run is gated by a physical GPIO start button: press once to start (with a live annotated camera preview shown beforehand for setup), press again to stop. All corner timing, speeds, and ROI thresholds are defined as named constants at the top of the file so the three main knobs turn delay, turn duration, and turn speed can be re-tuned in that order after each battery charge, since a fresher battery turns the robot faster



PREVIEW



DECISION MAKING FLOWCHART

Obstacle Challenge — Overall Strategy

Unlike the Open Challenge, where the robot's primary objective is to remain centered between the walls while detecting corner markers, the Obstacle Challenge requires the robot to make decisions based on the position and color of traffic-sign pillars. The obstacle program follows the same state-machine philosophy used in the Open Challenge, separating the run into independent stages that can each be tested and tuned individually. This modular structure made debugging significantly easier, since each behavior could be verified before being integrated into the complete navigation sequence.

Region-of-Interest Based Detection

Rather than processing the entire camera image every frame, the obstacle program divides the image into several Regions of Interest (ROIs). Each ROI is responsible for detecting a specific feature, such as nearby walls, approaching traffic signs, or the parking area. Restricting image processing to these smaller regions reduced computational load while also improving reliability by eliminating unnecessary parts of the image that could introduce noise into the detection process.



Traffic Sign Recognition and Avoidance

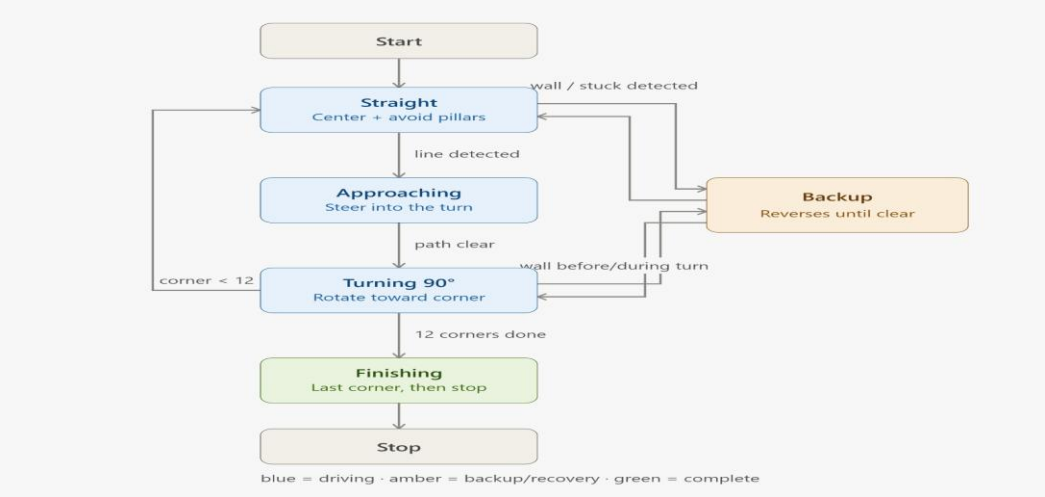
Traffic-sign pillars are detected using color segmentation in HSV color space. Once a valid red or green pillar is identified inside its corresponding ROI, the software determines both its color and its relative position within the camera frame. From this information, the program decides on which side of the obstacle the vehicle must pass while continuously monitoring its position until the maneuver is complete. Steering corrections are updated throughout the maneuver to compensate for small positioning errors and ensure that the robot safely clears each obstacle before returning to the center of the lane.

Dynamic State Transitions

Instead of relying on one continuous driving algorithm, the robot transitions between multiple states depending on what it observes. During normal driving it remains centered using the camera-based wall detection system. Once an obstacle is detected, control is temporarily transferred to the obstacle avoidance routine. After the pillar has been successfully passed, control returns to the normal wall-centering algorithm. This separation of responsibilities simplified software development and prevented different navigation routines from interfering with one another.

Parking Procedure

After completing the required laps, the software switches to a dedicated parking state. Using the camera together with the previously determined field configuration, the robot approaches the parking area while continuously correcting its alignment. Steering commands are gradually reduced as the vehicle enters the parking space to minimize oscillations and improve positioning accuracy. Once the parking maneuver is completed, the drive motor is stopped and the program terminates the run.


Start
Straight
Center + avoid pillars
wall / stuck detected
line detected
Approaching
Steer into the turn
Backup
Reverses until clear
path clear
wall before/during turn
corner < 12
Turning 90°
Rotate toward corner
12 corners done
Finishing
Last corner, then stop
Stop
blue = driving · amber = backup/recovery · green = complete

Systems Thinking & Engineering Decisions

Several of our early software iterations (multiple obstacle-detection and drive-recovery script versions) were attempts to compensate in code for what turned out to be a hardware problem weight distribution. Once we identified the pattern, we chose to invest time in a physical redesign (sliding battery plate) rather than continue tuning software around a chassis working against us.

Diagnosing Hardware Problems Instead of Patching in Software

Several of our early software iterations (multiple obstacle-detection and drive-recovery script versions) were attempts to compensate in code for what turned out to be a hardware problem weight distribution. Once we identified the pattern, we chose to invest time in a physical redesign (the sliding battery/powerbank plate) rather than continue tuning software around a chassis working against us.

Isolating Power Rails by Design

We deliberately kept the 5V logic supply (Raspberry Pi 5, camera, ultrasonic sensors) completely separate from the drive-side battery pack, rather than running everything off one supply. A shared rail would have been simpler to wire, but motor current draw and switching noise on a shared line risks browning out or resetting the Pi mid-run a failure mode that would be far harder to diagnose than a wiring inconvenience. We accepted the extra wiring complexity in exchange for electrical reliability.

Camera as Primary Sensor

Ultrasonic as an Independent Safety Net Navigation relies on the camera rather than the ultrasonic sensors because it gives richer information (wall coverage on both sides at once, plus color classification for corners and pillars) than a distance sensor can. The trade-off is that the camera is sensitive to lighting conditions. Rather than trying to make the camera handle every edge case, we kept the two HY-SRF05 ultrasonic sensors as a completely independent, lighting-agnostic safety layer that can override the camera-based logic at any time accepting some redundancy in exchange for not having a single point of failure.

Resuming Interrupted Turns Instead of Restarting Them

When the front-wall safety check interrupts a turn in progress, the code remembers exactly how much turn time was left and resumes for that remaining duration, rather than restarting the full turn from zero. This was a deliberate choice to avoid over-

rotating past the intended exit angle after a recovery a simpler implementation would have just re-run the full turn timer, but that would compound with every interruption.

Constraints

We Designed Within Some elements of the design were fixed constraints rather than choices: the single drive motor turning a solid rear axle is required by the competition's rules on mechanically-connected driving wheels, not a trade-off we made ourselves. Recognizing which parts of the design were fixed constraints and which were genuinely open decisions helped us focus our redesign effort on the things we could actually change chassis layout, weight placement, camera position, and wheel material.

The Wheel Material Decision

For the wheels, we chose to reprint using the same PLA filament at a higher nozzle temperature rather than switching to TPU/rubber. TPU wasn't available to us at the time, and we were working against a tight deadline, so reprinting with a material we already had on hand and knew how to tune was the faster path to a working fix.

Failure Modes Encountered

Overheating. The Raspberry Pi 5 ran hot enough under sustained camera processing load to risk throttling. Fixed by moving the cooling fan to sit on top of the Pi 5 instead of underneath it.

Loss of traction. Addressed by the wheel material change described above and in Section 2 the higher-temperature PLA print gave a softer, more rubber-like surface with noticeably better grip.

Camera calibration. Getting the camera to reliably detect the right colors (for wall/line and pillar detection) was one of the most time-consuming parts of the build. Getting thresholds that held up consistently took a lot of iteration this is reflected in the several vision-debugging scripts in our codebase used purely to tune and verify color detection during development.

Compute performance. The Raspberry Pi 5 4GB board we started with was too slow for our camera-processing pipeline and introduced noticeable lag. Finding an 8GB board that was both in stock and within budget was difficult on its own, but once sourced, the upgrade made a significant difference the 8GB Pi 5 runs our vision and control loop noticeably faster, with the lag we'd been fighting on the 4GB board gone.

Reproducibility & GitHub Repository

All code, CAD files, and this journal are maintained in our public GitHub repository. The repository README documents the module breakdown, how each module maps to the vehicle's electromechanical components, and the steps to build and flash the code.

https://github.com/CharbelKhalife/Mindscape-WRO2026-FUTURE-ENG

Testing Log

This section tracks the actual iteration path behind our final Open Challenge code and design including the strategies that didn't work to show how we arrived at the version described in Sections 2-5.

Camera Color Calibration

Before any driving strategy could work, we had to get the camera reliably detecting the right colors. This took a lot of back-and-forth adjusting threshold values by hand an early sign of how sensitive the whole approach would be to lighting and calibration, which is why our final codebase keeps color thresholds in dedicated configuration files rather than hard-coded in the driving logic.

Attempt 1 — Dynamic Line-Fitting

Our first Open Challenge strategy tried to draw a dynamic line on the camera preview and have the robot steer to follow it, recalculating the line each frame as the robot moved. In practice, fitting an accurate equation for that line in real time turned out to be extremely difficult, and the result was never accurate enough to steer by reliably. We moved on from this approach.

Attempt 2 — Early ROI Strategy + Camera FOV Investigation

At the same time, we were still dealing with the Version 1 weight-distribution and design problems described in Section 2, which made isolating any single problem harder. We tried an early version of our current ROI-based approach, but with far fewer regions/conditions on screen than the final version and it simply didn't work reliably. Our first guess was that the camera's limited field of view meant it wasn't seeing enough of the wall to make good decisions. We looked for a replacement camera, but nothing fit our budget and deadline. We then tried improvising a fisheye attachment (similar to a door peephole viewer) to widen the field of view cheaply this didn't produce a usable image for our detection logic, and we abandoned the idea.

Attempt 3 — Dual-Ultrasonic PID Wall-Following

We then set the camera-based approach aside and tried a strategy based purely on the two ultrasonic sensors, using a PID controller on the distance readings from both sides to center the robot between the walls. After spending significant time tuning it, this approach never stabilized, and our hardware/weight-distribution problems were still unresolved in parallel. This combination is what pushed us to commit to the physical redesign covered in Section 2.

After the Redesign — Returning to the ROI Strategy

Once the redesign was in place, overall behavior improved significantly, but we kept trying to make the ultrasonic PID approach work for a while longer before eventually giving up on it. We returned to the ROI/box-based camera strategy by this point our strongest lead helped by the camera's new, improved position on the chassis. From there we improved it incrementally: adding more regions and conditions one at a time (front-wall halves, pre-corner checks, stuck detection) until the behavior became reliable. Two problems remained even at this stage. First, the camera feed and overall control loop were laggy, which we traced to the Raspberry Pi 5 4GB board being underpowered for the processing load. Finding an 8GB board in stock and within budget took real effort, but after swapping it in, the lag was gone. Second, with the new hardware in place, we went through another full pass of recalibrating color thresholds and ROI parameters arriving at the working version described in Sections 2 through 5.

Obstacle Challenge Development

Development of the Obstacle Challenge followed a very similar path to our work on the Open Challenge. Our first idea was to extend the dynamic curve-fitting approach that we had originally investigated for wall following. The intention was to mathematically estimate the ideal path around each traffic-sign pillar and continuously steer along that calculated curve. Although the concept appeared promising, obtaining a stable and sufficiently accurate curve in real time proved extremely difficult. Small variations in camera detection produced large changes in the calculated path, making the robot's motion inconsistent. After several iterations, we concluded that this approach would not provide the reliability required for competition.

Transition to the ROI-Based Strategy

Following the unsuccessful curve-fitting approach, we shifted our attention to a Region-of-Interest based solution similar to the strategy that had proven successful in the Open Challenge. Instead of attempting to compute a complete path mathematically, we divided the camera image into multiple regions, each responsible for detecting a specific element of the course. This greatly simplified the decision-making process while also reducing the amount of image data that had to be processed every frame. The resulting software became easier to understand, debug, and improve.

Incremental Development

Rather than implementing the complete obstacle algorithm at once, we built it progressively. We first verified basic wall-centering behavior before adding traffic-sign detection. Once reliable obstacle detection had been achieved, we gradually introduced additional conditions, recovery behaviors, and safety checks. Every new feature was tested independently before being integrated into the complete program. This incremental approach allowed us to identify the source of unexpected behavior much more quickly than attempting to debug a fully completed system.

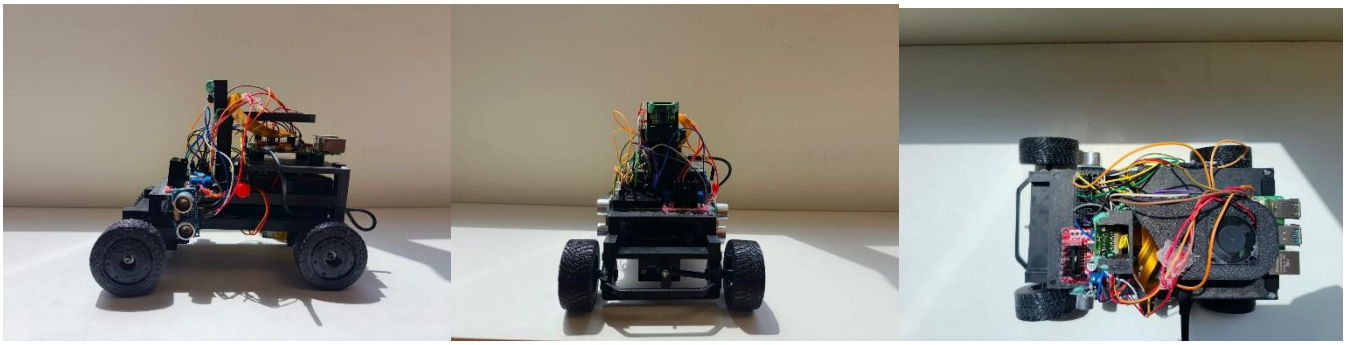
Steering Mechanism Failure

During testing we encountered an unexpected mechanical problem unrelated to the software itself. Our steering mechanism struggled to rotate sufficiently toward the left under certain conditions, producing inconsistent left turns despite the software issuing the correct commands. Since redesigning the steering system was not practical within the available time, we implemented a software workaround. Whenever a significant left steering command was required, the robot briefly reversed before steering again. This short backward motion relieved the mechanical load on the steering linkage, allowing the wheels to reach the desired angle before the robot continued moving forward. Although originally introduced as a temporary solution, extensive testing showed that it consistently improved the reliability of left turns and therefore remained part of our final implementation.

Final Validation

After integrating all navigation states, obstacle detection routines, recovery behaviors, and the steering workaround, the robot underwent repeated full-course testing under different lighting conditions and randomized field configurations. The primary objective of these tests was not only to improve success rate but also to ensure that modifications made to one behavior did not negatively affect another. Through repeated iteration, parameter tuning, and validation, the obstacle challenge software gradually evolved into the final version described throughout this journal.

Appendix A

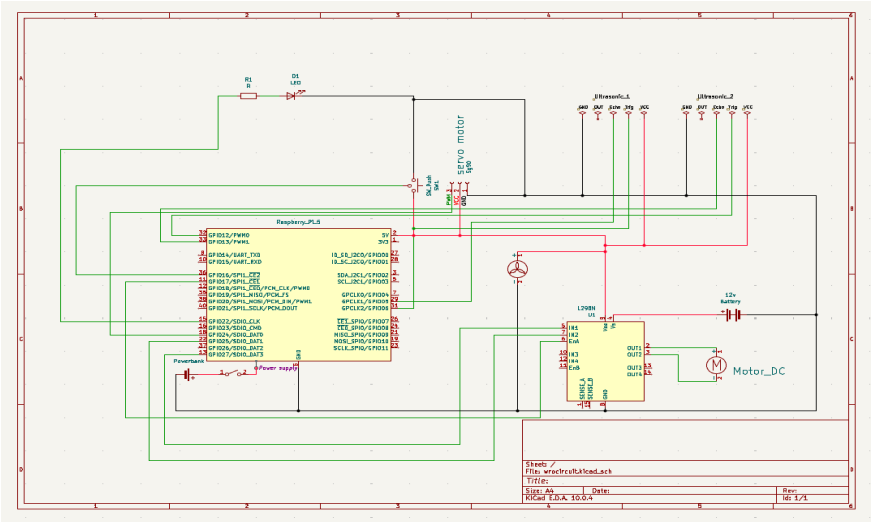


Robot Photos



Team Picture

Appendix B: Wiring Diagram



Appendix C: Video Links

Obstacle Challenge : ***https://youtube.com/shorts/zTeuM2oyFTw?feature=share***
Open Challenge: ***https://youtube.com/shorts/dPOwzluGnl0?feature=share***